
Playing Through an AI Incident

Sofia G. Gallego
Cosmoventures, Paris, France

Abstract

Little Agent Lab is a small browser game in which the player assembles a team of AI agents to complete a task while keeping a private item from leaking. The player chooses who joins the team, decides how much access each agent receives, and then runs the plan and observes the outcome directly, rather than being told what happened. Three missions are implemented so far; the final one presents two candidate safeguards, only one of which actually functions, and the player can only tell them apart by running the plan and reading the resulting trace. The motivation is early, informal exposure to ideas such as oversight and verification, for an audience — particularly children — that will otherwise mostly encounter these ideas as abstract prose. At present the project is best described as a proof of concept for this mechanic rather than a finished, engaging game; this report describes the design, what has been verified so far, and what remains to make it one.

1 Introduction

Public explanations of AI oversight — what it means for a system to be monitored, what a permission actually covers, why a safeguard can exist and still fail to work — are typically written for readers who already work in the field. This leaves a gap for a general audience, including the people who will grow up using these systems without ever having built one. This project addresses that gap as a communication exercise rather than a technical one: instead of explaining these ideas in prose, it puts a small, self-contained version of them in front of the player as something to do.

What exists at present is a working outline of that idea rather than a finished game. The player assembles a team of fictional agents, sets their access, and runs the plan. The final of three missions makes the underlying point directly: two helper cards offer to guard a gate, but only one of them actually blocks anything, and nothing in their description says which. The player finds out by running the plan and reading what happened. This mechanic — a safeguard is only as good as what it does, not what it is labeled — is the central idea the project is built to communicate; whether the current implementation is engaging enough to hold a child’s attention is a separate question this report does not yet answer.

2 Related Work

Games that teach a security habit by first letting the player fail at it are well established outside AI safety. *Anti-Phishing Phil* [3] put players through a simulated inbox and measured real gains in phishing-detection accuracy afterward. *Control-Alt-Hack* [1] used a card game to introduce non-specialists to the idea that a system can appear secure while one of its components silently fails — close in spirit to what the Message Relay mission does here, though narrower in scope and specific to AI agents. Within AI safety, the “specification gaming” examples collected by Krakovna et al. [2] document real cases in which a system satisfied a check on paper while missing its intent entirely; that is the phenomenon this project dramatizes, in miniature and entirely fictionally.

No existing browser game was found that makes this specific idea — a safeguard that is present but does not work — the central, escalating mechanic for a general audience. The claim of novelty here is narrow: not the broader genre of security-awareness games, but this particular framing, built small enough to implement and check within a short timeframe.

3 Methodology

The game runs entirely in the browser, with no accounts and no live model anywhere in the loop. Each agent is a small, hand-written function: given a team and a gate setting, it produces the same outcome every time. This was a deliberate choice — the goal is to communicate a concept clearly, not to simulate an actual unreliable model, which is a substantially different and larger problem.

The player selects a builder agent, optionally adds up to two helper agents, decides whether the builder may leave the workshop through a gate, and runs the plan. The outcome plays out as a short, literal trace of events, using statements such as `lens.deny(payload.private === true)`, so that what happened is something the player can read directly rather than something the interface merely asserts.

Three missions are implemented, unlocked in sequence:

- **Weather Lantern.** Two strategies succeed: pairing the builder with a helper whose route never touches the private item, or opening the gate and pairing it with an inspector that checks what leaves. This introduces capability separation as one valid approach among several.
- **Garden Delivery.** No side route is available; the builder must leave through the gate, so the only winning configuration pairs that route with a functioning inspector. The mission illustrates that caution alone does not always complete the task, and that the safe path is sometimes the risky path executed correctly.
- **Message Relay.** Two helper cards offer to guard the gate, but only one actually blocks anything, and their labels do not indicate which. The player must run the plan and read the trace to find out. This is the mission the project is built around.

4 Results

Each mission's logic is implemented as a small branching function, and every branch — every win and every loss condition — is covered by an automated test: 27 in total, all passing, verifying the correct outcome for every meaningfully distinct team configuration. Separately, the built application was driven with a headless browser through the goal screen, a complete mission win, a mission-unlock transition, and a second mission's distinct card set, checking for runtime errors at each step; none were found.

These checks establish that the game behaves as designed and does not break under normal use. They do not establish that the intended lesson is communicated to a real player. No testing with actual users, adult or child, has been carried out, and this is the most significant gap in the present evaluation.

5 Discussion

The design rests on the assumption that watching one's own incorrect guess fail communicates something a paragraph of explanation does not, because the player forms the conclusion directly rather than being handed it. This is consistent with the findings reported for phishing-awareness games, but has not been measured here, for this game, this age group, or this specific mechanic, and should be read as a working assumption rather than a result.

In its current form, the project is a working outline of the mechanic rather than a game a child would be expected to want to keep playing. The most immediate next step is therefore not additional content

but engagement: pacing, character and visual design, and feedback that rewards the player rather than simply marking an answer correct. Beyond that, planned extensions include additional missions so the underlying idea recurs in different forms, a version of Message Relay in which the non-functioning safeguard's tell changes between runs so the lesson cannot simply be memorized, and a playtest with children, the intended audience this project has not yet been shown to.

Limitations and Dual-Use Considerations

Limitations. This is a proof of concept for a single mechanic, not a finished game, and it is not yet judged to be engaging enough for a child to want to play. No testing with real players has been conducted, so every design decision reflects the author's own judgment rather than external validation. Three missions are sufficient to demonstrate the mechanic but not to establish that the underlying lesson generalizes. The in-game trace is considerably more legible than any real system's actual safeguards, and a player could come away overestimating how quickly or easily real-world verification can be done. Implementation was AI-assisted throughout (Claude Code), which is disclosed here in line with the sprint's guidance on accurately describing contributions.

Dual-use. The dual-use risk is assessed as low. Every agent name, destination, and payload in the game is fictional and does not correspond to any real API, protocol, or system. The one mechanic that resembles a "bypass," Message Relay's non-functioning guard, is a fixed, pre-authored branch selected from two labeled cards rather than a generative or discoverable exploit, and nothing learned in the game is applicable against a real system. No model was jailbroken and no organization's real systems were involved in this work.

References

- [1] Denning, T., Lerner, A., Shostack, A., & Kohno, T. (2013). Control-Alt-Hack: A Card Game for Computer Security Awareness and Education. *Proceedings of the 2013 ACM SIGSAC Conference on Computer & Communications Security (CCS '13)*.
- [2] Krakovna, V., Uesato, J., Mikulik, V., Rahtz, M., Everitt, T., Kumar, R., Kenton, Z., Leike, J., & Legg, S. (2020). Specification gaming: the flip side of AI ingenuity. *DeepMind blog*.
- [3] Sheng, S., Magnien, B., Kumaraguru, P., Acquisti, A., Cranor, L.F., Hong, J., & Nunge, E. (2007). Anti-Phishing Phil: The Design and Evaluation of a Game That Teaches People Not to Fall for Phish. *Proceedings of the 3rd Symposium on Usable Privacy and Security (SOUPS '07)*.

Project source: <https://github.com/nefinia/little-agent-lab> (public, playable via the repository's GitHub Pages workflow).